ATELIER PRATIQUE MLOps

Intégration de Git + DVC + MLflow dans un workflow reproductible

Nom et prénom : **Zakariae CHELLE** Groupe : **5IASDT G1**

Module : **Déploiement des applications intelligentes** | Dépôt GitHub : *https://github.com/ZAKARIAE48CHELLE/MLOPS-TP-1*

Ce compte-rendu documente la mise en place complète d'un cycle de vie MLOps intégrant le versioning du code source (Git), le versioning et l'orchestration des pipelines de données (DVC), et le suivi rigoureux des expérimentations scientifiques (MLflow) sur le dataset Iris.

Partie 1 — Architecture et Rôles des Outils MLOps

L'industrialisation des modèles d'apprentissage automatique requiert une stricte séparation des responsabilités entre le code, les données volumineuses et les métadonnées expérimentales :

Composant	Responsabilité principale	Fichiers suivis	Avantage clé
Git + GitHub	Versioning du code source, scripts et configurations	src/*.py, params.yaml, dvc.yaml, .dvc/	Traçabilité fine, branches et collaboration
DVC	Gestion des données, modèles volumineux et DAG	data/raw/iris.csv.dvc, models/model.pkl, cache	Reproductibilité sans encombrer Git
MLflow Tracking	Journalisation des métriques, paramètres et artefacts	sqlite:///mlflow.db, runs RF et KNN	Comparaison scientifique et aide à la décision

> ***Rappel méthodologique :*** *Git gère les fichiers texte légers (code, configs) ; DVC gère les fichiers volumineux (datasets, modèles sérialisés) via des pointeurs MD5 ; MLflow consigne l'historique des expérimentations et leurs métriques.*

1.1 Arborescence normalisée du projet

```
MLOPS-1/
├── .dvc/                 # Configuration DVC et remote de stockage (../dvc-storage)
├── .github/workflows/    # Pipeline CI/CD automatisé (GitHub Actions mlops.yml)
├── data/raw/             # Données sources (iris.csv, iris.csv.dvc)
├── models/               # Modèles entraînés suivis par DVC (model.pkl)
├── reports/              # Métriques (metrics.json) et visualisations
├── src/                  # Code source (prepare_data.py, train.py)
├── params.yaml           # Paramètres configurables de l'entraînement
├── dvc.yaml / dvc.lock   # Définition et état figé du pipeline reproductible
├── load_model.py         # Script d'inférence et de rechargement
└── requirements.txt      # Dépendances logicielles du projet
```

Partie 2 — Réalisation des Travaux Pratiques (TP 1 à TP 4)

2.1 TP 1 — Gestion du code source avec Git

Un historique de commits rigoureux a été établi avec une séparation nette des branches de travail :

- **Branche** `main` : branche principale stable recevant les versions validées.
- **Branche** `develop` : branche d'intégration pour le développement et l'orchestration du pipeline.
- **Tags Git :** **`v1.0`** (version Random Forest 100 arbres) et **`v2.0`** (version Random Forest 200 arbres).

2.2 TP 2 — Versioning des données avec DVC

Les fichiers de données ont été isolés sous le contrôle de DVC sans polluer l'historique Git :

```
# Extraction et génération du dataset
python src/prepare_data.py

# Ajout sous contrôle DVC
dvc add data/raw/iris.csv
dvc add data/raw/winequality-red.csv   # Deuxième dataset (TP 2)

# Configuration du stockage distant local
dvc remote add -d storage ../dvc-storage
dvc push
```

> ***Observation :*** *La commande dvc push transfère les données vers le remote storage, tandis que Git ne conserve que le pointeur data/raw/iris.csv.dvc contenant l'empreinte MD5.*

2.3 TP 3 — Tracking des expérimentations avec MLflow et intégration DVC

Trois configurations de modèles ont été exécutées et tracées dans la base de données SQLite (sqlite:///mlflow.db) sous l'expérience TP_Iris_Classification :

Nom du Run	Modèle	Hyperparamètres	Accuracy	Precision	Recall	F1-Score
RandomForest_n_100_depth_5	Random Forest	n_est=100, depth=5	0.9333	0.9333	0.9333	0.9333
RandomForest_n_200_depth_8	Random Forest	n_est=200, depth=8	0.9000	0.9024	0.9000	0.8997
KNN_k_5	K-Neighbors	k=5, weights=uniform	1.0000	1.0000	1.0000	1.0000

> ***Analyse comparative :*** *Sur cet échantillon stratifié du dataset Iris, le modèle KNN k=5 obtient un score parfait (1.0000). Le passage de 100 à 200 arbres sur Random Forest n'apporte aucun gain d'accuracy, confirmant qu'augmenter la complexité d'un modèle sur un petit dataset peut induire un léger sur-apprentissage.*

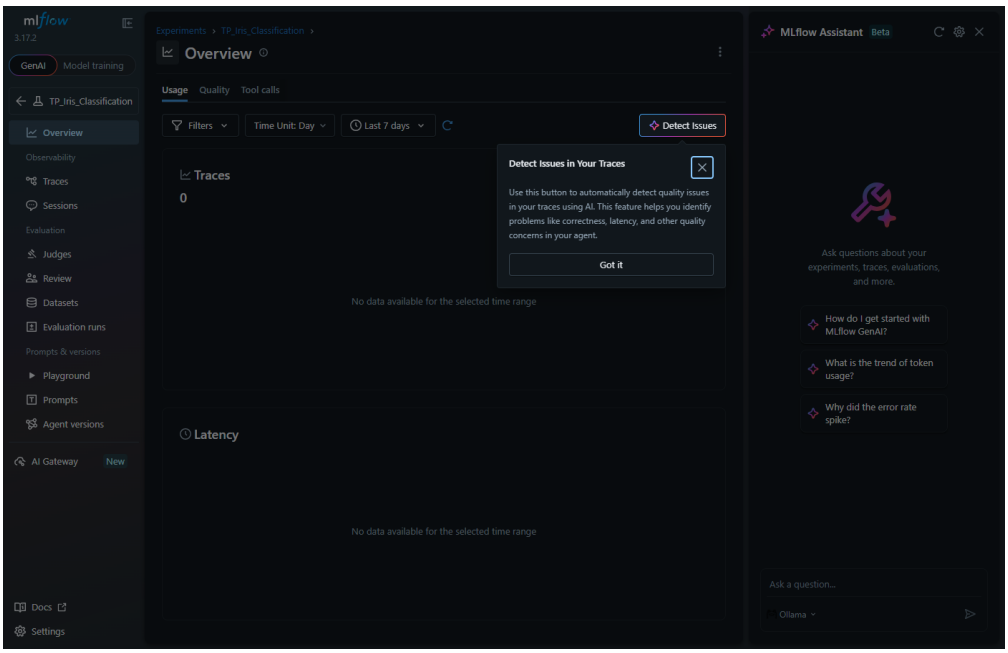


Figure 1 — Interface Web MLflow : tableau de bord de l'expérience TP_Iris_Classification présentant les runs et leurs métriques d'évaluation

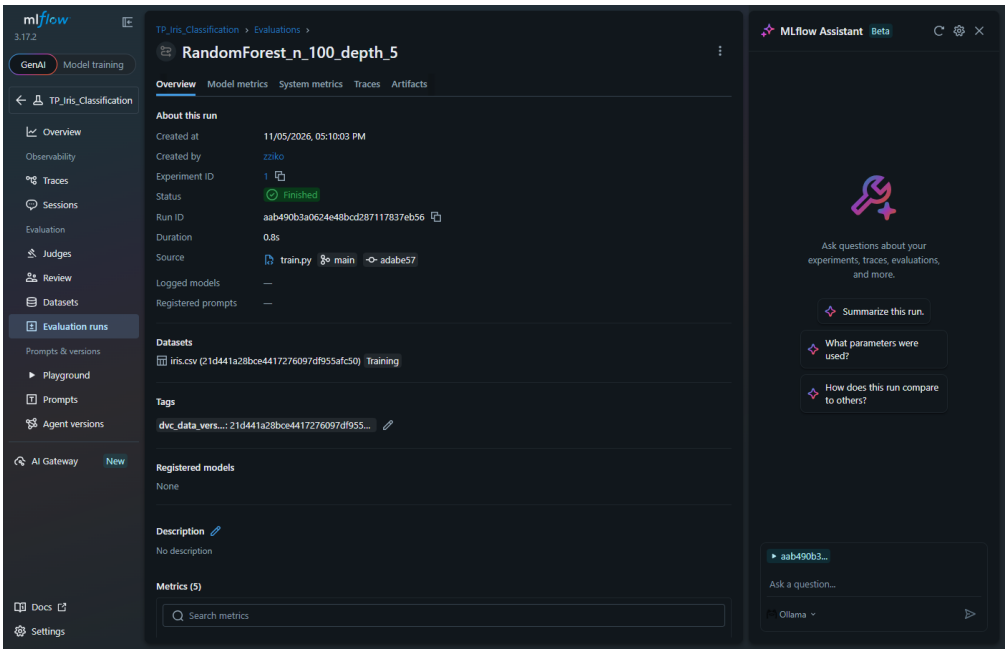


Figure 2 — Détail d'un run MLflow montrant la traçabilité de l'empreinte de données DVC (dvc_data_md5), le commit Git et les hyperparamètres

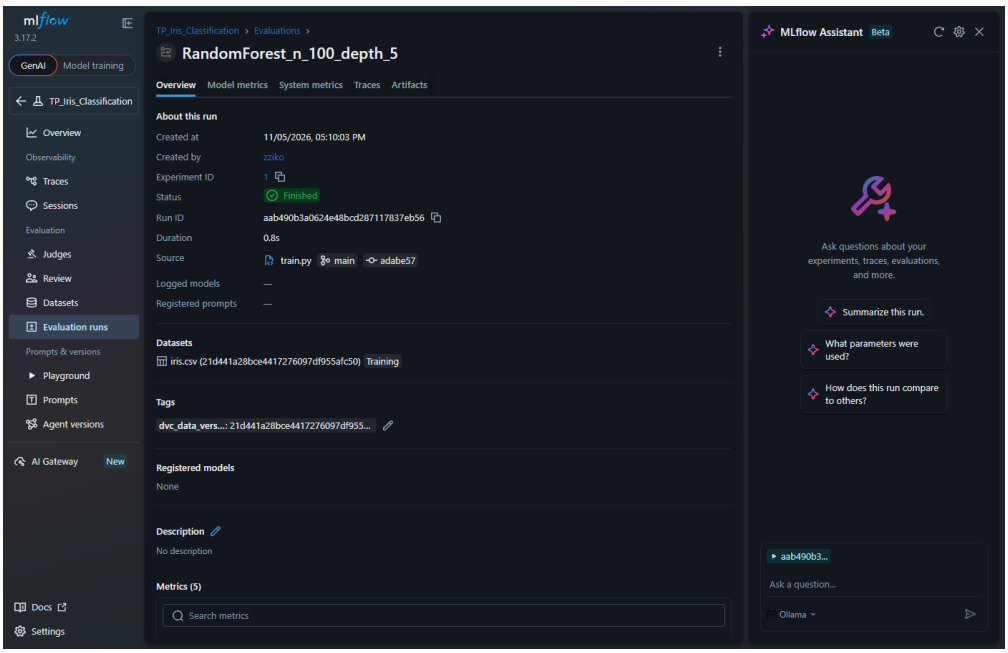


Figure 3 — Section Artefacts de MLflow affichant le dataset brut (data/iris.csv), le modèle entraîné (model/model.pkl) et la matrice de confusion

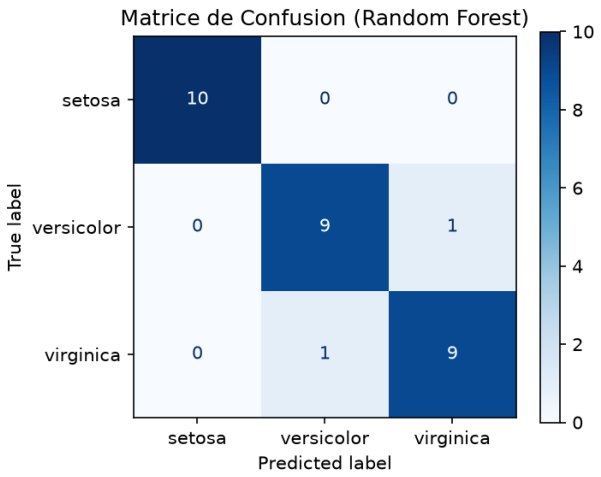


Figure 4 — Matrice de confusion du modèle Random Forest générée et enregistrée dans les artefacts MLflow

2.4 TP 4 — Pipeline MLOps intégré (DVC + MLflow)

L'enchaînement des étapes est orchestré par le fichier dvc.yaml qui surveille les dépendances et produit les métriques et artefacts :

```
# Exécution déterministe du pipeline
dvc repro

# Visualisation du graphe orienté acyclique
dvc dag

# Affichage tabulaire des métriques calculées
dvc metrics show
```

> ***Bénéfice de reproductibilité :*** *Si ni les données sources, ni les scripts, ni params.yaml n'ont changé, DVC réutilise instantanément le cache local (Stage is cached - skipping run).*

Partie 3 — Réponses aux Questions de Réflexion (Section 16)

1. **Quelle est la différence entre Git et DVC ?**

   Réponse : Git est un système de contrôle de versions conçu pour suivre des fichiers texte légers (scripts Python, fichiers de configuration). Il calcule des différences ligne par ligne. Il n'est pas adapté au stockage de fichiers volumineux ou binaires.

   DVC (Data Version Control) est conçu spécifiquement pour gérer les fichiers volumineux (jeux de données, modèles binaires). Il stocke les données dans un cache externe ou un stockage distant et ne confie à Git qu'un petit fichier pointeur textuel (.dvc) contenant une empreinte MD5.

   > ***Pour comprendre :*** *Git suit la logique du code ; DVC suit l'état et la provenance des données volumineuses sans alourdir le dépôt.*

2. **Pourquoi éviter de stocker un dataset volumineux directement dans Git ?**

   Réponse : Stocker des fichiers volumineux dans Git entraîne une explosion irréversible de la taille du dossier .git, car chaque version modifiée conserve une copie complète de l'objet.

   Cela ralentit considérablement les opérations réseau (clone, pull, push), enfreint les limites imposées par les forges logicielles (ex. limite de 100 Mo sur GitHub), et ne permet pas d'effectuer de comparatifs textuels pertinents sur des formats binaires.

3. **Quelle est la différence entre dvc add et dvc repro ?**

   Réponse : dvc add est une commande manuelle de suivi statique d'un fichier ou dossier de données. Elle calcule l'empreinte de hachage, place les données dans le cache DVC et génère le fichier .dvc correspondant.

   dvc repro est une commande d'orchestration dynamique. Elle lit le fichier de pipeline dvc.yaml, examine les dépendances modifiées et réexécute automatiquement les étapes nécessaires en mettant à jour le fichier dvc.lock.

4. **Quel est le rôle du fichier dvc.lock ?**

   Réponse : Le fichier dvc.lock enregistre l'état résolu et figé du pipeline après exécution. Il consigne les empreintes MD5 exactes des dépendances (fichiers sources et données), les valeurs des paramètres utilisés, et les empreintes des sorties produites.

   Versionné dans Git, dvc.lock garantit la reproductibilité absolue : tout collaborateur peut réinstancier exactement les mêmes sorties avec dvc checkout sans recalcul inutile.

5. **Comment MLflow permet-il de comparer deux expérimentations ?**

   Réponse : MLflow centralise tous les runs d'une même problématique sous une expérience unique (TP_Iris_Classification). Dans son interface Web (mlflow ui), l'utilisateur peut sélectionner plusieurs runs et cliquer sur Compare.

   MLflow génère automatiquement un tableau comparatif côte à côte des hyperparamètres et des métriques, complété par des graphiques interactifs (diagrammes en barres, scatter plots, courbes de dispersion) pour identifier le modèle optimal.

6. **Comment retrouver les données et le modèle associés à un commit Git précis ?**
   Réponse : On se positionne tout d'abord sur le commit Git cible via la commande git checkout <commit_hash> (ou via un tag tel que v1.0). Cette commande restaure l'état exact des fichiers pointeurs (.dvc, dvc.yaml, dvc.lock).

   Il suffit ensuite d'exécuter dvc checkout (ou dvc pull) pour que DVC télécharge et replace dans l'arborescence locale les données d'entraînement et le modèle binaire model.pkl qui correspondent rigoureusement à ce commit.

7. **Pourquoi la reproductibilité est-elle importante dans un projet d'intelligence artificielle ?**
   Réponse : La reproductibilité est fondamentale pour l'auditabilité et la conformité réglementaire : elle permet d'expliquer et de justifier les prédictions d'un modèle déployé.

   Elle est indispensable pour le débogage (reproduire une anomalie constatée en production), la collaboration saine au sein des équipes Data, et l'intégration continue (CI/CD) permettant d'entraîner et tester les modèles de manière déterministe.

8. **Quelle est la différence entre le versioning des modèles avec DVC et leur suivi avec MLflow ?**
   Réponse : DVC versionne le modèle comme un artefact binaire au sein du pipeline de données : il garantit que le fichier physique model.pkl est lié de manière déterministe aux données sources et au code via dvc.lock.

   MLflow suit le modèle sous un angle scientifique et décisionnel : il conserve le contexte complet (métriques d'évaluation, paramètres, courbes de perte, signature des entrées/sorties) et fournit un Model Registry pour piloter les étapes de déploiement (Staging, Production).

> ***Complémentarité MLOps :*** *DVC garantit la reproductibilité de fabrication du modèle ; MLflow assure son évaluation scientifique et sa gouvernance opérationnelle.*

Partie 4 — Automatisation et Intégration Continue (GitHub Actions)

Un pipeline d'intégration continue a été implémenté dans le fichier .github/workflows/mlops.yml afin de valider automatiquement la reproductibilité à chaque modification poussée sur GitHub :

```
name: MLOps Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
  train:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: dvc repro
      - run: dvc metrics show
```

Conclusion

Cet atelier démontre l'efficacité de la combinaison Git + DVC + MLflow pour répondre aux exigences industrielles du MLOps. Grâce au pipeline automatisé, chaque version de modèle est associée à un commit Git précis, une version de données immuable sous DVC et un suivi expérimental exhaustif dans MLflow, garantissant une reproductibilité et une gouvernance totales.